SHH Revisited: Don't Just Run Models, Reason About Them

Jeff Shrager

Draft, November 2026

Draft for Comment[1]

Abstract

In 1988, Tad Hogg, Bernardo Huberman and I proposed that the power law of practice need not be built into a learning mechanism: it could emerge from the dynamics of search in a problem space that learning makes more densely connected, and the same dynamics could make practice hurt, a problem-solving fan effect. The paper supported this with simulations of a simple "bit game" and with an analytic theory based on a random walk. Here I revisit that work by formalizing it in the Lean proof assistant, so that what the theory entails is checked by machine rather than read off curves. Most of the 1988 mathematics survives: the expected-steps formula is correct exactly as printed, and its drift and diffusion regimes hold, one of them in a sharper form. But two of the paper's conclusions do not follow from its own theory. With no decision procedure, the idealized model shows a fan effect that the paper says is absent; and with a perfect decision procedure the "power-law decay" it derives is logarithmic, a curve with every qualitative signature of a power law over the range plotted, and provably not one over long ranges. In the bit game itself, exact results reproduce the 1988 simulations' pattern, which locates the first discrepancy precisely: in the idealization that connects the game to the theory, not in the game. None of these findings needed new mathematics. They needed the theory written down precisely enough that its consequences could be checked.

1 Introduction

The power law of practice is one of the most robust regularities in the psychology of learning: the time to perform a task falls as a power of the number of times it has been performed [1–3]. In 1988 the leading accounts obtained it in quite different ways. ACT* obtained it from a strengthening mechanism that itself follows a power law [4]; Newell and Rosenbloom obtained it from the chunking of subgoals [3, 5]. Tad Hogg, Bernardo Huberman and I proposed a third account [6]. Problem solving is search through a problem-space graph; learning adds edges to the graph (new operators) and can improve the procedure that chooses among them. Neither change mentions a power law. Yet their interaction, under particular conditions, yields curves that look like one, and under other conditions yields the opposite: learning that makes performance worse. We called that the problem-solving fan effect, by analogy with the memory fan effect [7], and connected it to the utility problem in machine learning [8, 9].

That paper did its reasoning in two ways. It simulated a small task, the bit game, and it derived an analytic theory, a one-dimensional random walk whose expected length depends on how

[1] This is a working draft. I welcome corrections, constructive challenges, and especially proposals for collaboration. Email me: `jshrager@gmail.com`.

densely the graph is connected and how good the decision procedure is. Each regime of the walk was connected to a phenomenon: drift toward the goal to improvement with practice, a balance of branching against the shortening of paths to the fan effect. In this sense the 1988 paper already had the explanatory shape I have since been arguing for in CogLean [10, 11]: mechanisms, together with stated conditions, entail phenomena, including conditions under which a phenomenon fails or reverses.

What it did not have was a way to check that its conclusions followed. The derivations were done by hand, the approximations were judged by eye, and the agreement between theory and simulation was read off log-log plots. So I have written the 1988 theory and the bit game in Lean [12, 13], where the definitions that run are the same objects that theorems are about, and every claim of the paper has been given a status: proved, sharpened, reinterpreted, or refuted (Table 3). The work is in a public repository [14]; every result below names the theorem that establishes it.

The outcome is a mixed verdict, which is the interesting kind. The mathematics of the walk is right. But the two conclusions the paper drew from it about practice do not follow: one is contradicted by the paper's own idealized model, and the other names the wrong curve. And the place where theory and game come apart can be located exactly.

2 The model

I restate the 1988 model briefly, as the formalization reads it; the full list of postulates, conditions and idealizations, with page references, is in `notes/POSTULATES.md`.

The bit game. A state is a vector of B bits. An operator is a pattern–result pair with don't-care positions: `?1?1?` → `?0?0?` applies to any state with 1s in the second and fourth positions and sets them to 0. Play begins with the $2B$ one-bit operators, which connect the states as a hypercube. A trial draws a start and a goal, and performance is the number of operators applied to reach the goal (`SHH.BitGame.apply`, `SHH.BitGame.initialOps`).

Learning. After each trial, the operator that solves that game in one step is added, with a don't-care wherever start and goal agree. For `01110` → `10101` it is `01?10` → `10?01` (`SHH.BitGame.chunk_paper`), which "connects two pairs of states": it applies to exactly two of the 32 states (`SHH.BitGame.chunk_paper_applies`). In general a learned operator takes its own start to its own goal (`SHH.BitGame.chunk_apply`).

The decision procedure. At a state, the applicable operators whose results are closest to the goal (by Hamming distance) are good, the rest bad; each bad one is discarded with probability r, and one operator is chosen at random from what remains. So $r = 1$ is a perfect procedure and $r = 0$ a random walk. The formalization computes the exact distribution over next states, with the probabilities of good and bad choices as binomial sums (`SHH.BitGame.stepDist`).

The theory. To analyze typical rather than particular problem spaces, the paper idealizes the graph as random, with n nodes and mean branching μ, and then as a tree: start and goal are about the diameter $D = \ln n / \ln \mu$ apart, and at each node one edge leads toward the goal and $\mu - 1$ away. The decision procedure leaves $(\mu - 1)(1 - r)$ effective bad choices, so a step goes toward the goal with probability

$$p_- = \frac{1}{1 + (\mu - 1)(1 - r)}. \tag{1}$$

The search becomes a walk on distance $0, \ldots, D$ from the goal (`SHH.GD.pOf`, `SHH.GD.DOf`). Everything in the theory depends on this idealization, and the formalization keeps it separate from the game, as an explicit postulate. That separation turns out to matter.

3 The theory, checked

The expected number of steps. The paper gives the expected number of steps s to reach the goal from distance D as its Eq. 2, obtained by "standard techniques" [15]. The formalization states the walk as a linear system for the expected steps from every distance (`SHH.Walk.Sys`), proves a closed form satisfies it (`SHH.Walk.term_sys`), and proves the system has only one solution (`SHH.Walk.sys_unique`); together these make the closed form the expected number of steps (`SHH.Walk.sys_eq_s`). The paper's Eq. 2 as printed,

$$s = \frac{2}{(2p_- - 1)^2}\left[p_-^2 + p_-(D-1) - \frac{D}{2} + \frac{(1-p_-)^{D+1}}{p_-^{D-1}}\right],$$

is exactly this function for every p_- other than 0 and $\frac{1}{2}$ (`SHH.Walk.eq2_printed`). The 1988 derivation was right.

One detail had to be decided before any of this could be stated: what the walk does at distance D, where the paper says only that it is "constrained to remain within distance D". The paper's own value for the symmetric case, $s = D^2$, settles it: that value holds if the walk is reflected back at D (`SHH.Walk.s_half`), and not if it may stay there, which gives $D^2 + D$ (`SHH.Walk.stay_half_sys`). A choice the paper left implicit is recoverable from what it reported.

The regimes. When steps toward the goal are more likely than steps away ($p_- > \frac{1}{2}$), the paper's Eq. 3 says the walk drifts to the goal in about $D/(p_- - p_+)$ steps. This holds as a two-sided bound: the expected steps lie within $2p_-(1-p_-)/(2p_- - 1)^2$ below $D/(2p_- - 1)$, a constant that does not grow with D (`SHH.Walk.drift_bounds`). When steps away are more likely, the paper's Eq. 4 gives the exponential term of Eq. 2, and the rest is a term only linear in D (`SHH.Walk.eq4_remainder`, `SHH.Walk.s_diffusion_form`). In every regime, more distance costs strictly more steps (`SHH.Walk.s_strictMono_D`).

4 What the theory entails, and what it does not

The paper then turned the walk into claims about practice. Learning adds edges, so μ grows; more edges shorten the diameter, which helps, and add bad choices, which hurts. Two of the paper's conclusions about which effect wins do not follow.

A fan effect where the paper says there is none. For a weak decision procedure the paper approximates Eq. 4 by its Eq. 5, $s \approx (2n/\mu)(1-r)^{\ln n/\ln \mu - 1}$, and concludes that "when $r = 0$ (choices made at random), increasing the number of edges reduces the time to solve the problem". The approximation replaces $(\mu-1)^{D-1}$ by $\mu^{D-1} = n/\mu$, dropping a factor $((\mu-1)/\mu)^{D-1}$ that is at most $e^{-(D-1)/\mu}$ (`SHH.GD.eq5_error_bound`). Where μ is small, which is where learning starts, that factor is far from one. Restoring it reverses the conclusion. With $n = 2^{16}$ nodes, where the diameter is an integer for $\mu = 2, 4, 16, 256$ (`SHH.GD.DOf_pow2`), the paper's own idealized model gives the expected steps in Table 1: at $r = 0$, raising the mean branching from 2 to 4 raises the expected

steps from 256 to 9,824, and only from $\mu = 16$ on do more edges help (`SHH.GD.fan_pow2_zero`). In the theory, there is a fan effect with no decision procedure at all.

μ	D	$r = 0$	$r = \frac{1}{2}$	$r = 1$
2	16	256	44.0	16
4	8	9,824	255.5	8
16	4	7,744	1,117.75	4
256	2	512	257†	2

Table 1: Expected steps in the paper's idealized model, $n = 2^{16}$, from the proved closed form (`SHH.GD.fan_pow2_zero`, `SHH.GD.fan_pow2_half`, `SHH.Walk.s_one`). †From the prototype (`tools/walk_check.py`), not certified.

A logarithm where the paper says power law. With a strong decision procedure the paper's Eq. 6 gives $s \approx \ln n / \ln \mu$ and concludes that "one obtains a power-law decay". At $r = 1$ the approximation is exact: the walk never steps away, so $s = D$ (`SHH.Walk.s_one`). To read this as a learning curve one needs a further condition the paper leaves implicit, that each trial adds a bounded number of edges, so that μ grows about linearly with trials. The curve is then $T(t) = \ln n / \ln(\mu_0 + ct)$ (`SHH.GD.curveGD`). It improves with every trial (`SHH.GD.curveGD_improves`), and its proportional gains shrink trial by trial (`SHH.GD.curveGD_diminishingRelative`), the signature that distinguishes a power curve from an exponential one [16]. Over trials 1–100 it lies within about 12% of a power law with exponent -0.29 (`tools/walk_check.py`). But it is not a power law. Its doubling ratio $T(2t)/T(t)$ tends to 1, so for any exponent and tolerance, approximate scale invariance fails beyond some trial (`SHH.GD.curveGD_not_powerLaw`). The curve is logarithmic. On a log-log plot over the one decade of Figure 1 of the 1988 paper, the difference is invisible.

This does not undo the 1988 argument against building the power law into the mechanism. It sharpens it. The graph-dynamic model does not build in a power law, and, at least in its perfect-decision regime, it does not produce one either. What it produces is a curve that cannot be told apart from one over the range anyone plotted.

5 The game itself

The theory's conclusions are about an idealization. Is the fan effect at $r = 0$ a property of search in the bit game, or of the idealization? The formalization can answer this exactly for small games, because the expected number of steps in the game is the solution of a finite linear system, and that solution is unique whenever the goal is reachable (`SHH.Markov.hit_unique`, proved by the maximum principle). So a table of values checked to satisfy the system is the expected number of steps.

On the three-bit game, adding one learned operator lowers the mean expected steps over all 56 games at $r = 0$ and at $r = 1$, and raises it at $r = \frac{1}{2}$ (Table 2; `SHH.BitGame.figure1_pattern_three`). The prototype finds the same for every one of the 32 operators that could be learned. This is the pattern of the 1988 simulations: improvement with no decision procedure and with a perfect one, degradation with a mediocre one. So the bit game does not show the fan effect at $r = 0$ that its idealization does. The discrepancy is in the idealization: a hypercube is full of short cycles, and the tree that the theory substitutes for it has none. (Individual games can still slow down at $r = 0$: `000` $\to$ `001` goes from 7 to 133/15 expected steps, `SHH.BitGame.chunk_slows_game`. It is the average that improves.)

Claim of the 1988 paper	Status
Eq. 2, the expected steps	proved, exactly as printed (`SHH.Walk.eq2_printed`)
$s = D^2$ when $p_+ = p_-$	proved; fixes reflection at D (`SHH.Walk.s_half`)
Eq. 3, drift	proved as a two-sided bound (`SHH.Walk.drift_bounds`)
Eq. 4, diffusion	proved as Eq. 2's exponential term (`SHH.Walk.eq4_remainder`)
Eq. 5 and "at $r = 0$ more edges reduce the time"	refuted in the paper's own idealized model (`SHH.GD.fan_pow2_zero`)
Fan effect at intermediate r	proved, theory (`SHH.GD.fan_pow2_half`) and game (`SHH.BitGame.figure1_pattern_three`)
Eq. 6 and "a power-law decay"	reinterpreted: logarithmic, not a power law (`SHH.GD.curveGD_not_powerLaw`)
Eq. 2 "produces the range of behaviors observed"	fails at $r = 0$ because of the tree idealization (`SHH.BitGame.figure1_pattern_three`)
$r = 1$ approximates optimal search	proved optimal on the initial operators (`SHH.BitGame.r1_moves_closer`); not optimal with learned ones (`SHH.BitGame.greedy_not_optimal`)

Table 3: The 1988 claims and their status. The full list is in `notes/CLAIMS.md`.

	$r = 0$	$r = \frac{1}{2}$	$r = 1$
initial operators	$58/7 \approx 8.286$	$128/35 \approx 3.657$	$12/7 \approx 1.714$
plus the chunk `00?` $\rightarrow$ `11?`	$2603/315 \approx 8.263$	≈ 3.682	$23/14 \approx 1.643$

Table 2: Mean expected steps over the 56 games of the three-bit game, exact (`SHH.BitGame.mean_r0_Base` and its five companions).

Two results about the perfect decision procedure complete the picture. With the initial operators, every move it can make brings the state one bit closer to the goal, for any number of bits, so it never searches or backtracks (`SHH.BitGame.r1_moves_closer`). Once operators have been learned, this fails: on the four-bit game, with the learned operator `0000` $\rightarrow$ `1111`, the game `0001` $\rightarrow$ `1110` has a three-move solution (`SHH.BitGame.three_step_path`) but the perfect procedure expects 15/4 moves (`SHH.BitGame.greedy_not_optimal`). Choosing the move that looks best by distance can miss the shortcut that learning created. The 1988 paper said as much, that its $r = 1$ procedure "only approximates optimal problem solving"; here is a case, exactly.

6 Discussion

The 1988 paper was mathematically careful, and most of its mathematics survives a machine check unchanged. What did not survive were two inferences from the mathematics to the psychology: an approximation applied where it is poor, and a curve called by the name of the curve it resembles. Neither is an unusual kind of mistake. Both are the kind a proof assistant catches almost incidentally, because it will not accept "approximately" without saying approximately what, over which range, and why.

The more interesting finding is about idealization. The 1988 paper moved from the bit game to a random graph to a tree to a walk in a few sentences, and treated the walk's behavior as the game's. In the formalization the game and the walk are separate objects, and the step between them is a postulate with a name. That is what made it possible to ask whether a conclusion of the theory

holds of the game, and to find one that does not. I think this is the general lesson. Idealizations are where theories make their least visible commitments; writing them down as first-class objects, rather than as a sentence of motivation, makes them something one can test.

It also says something about the 1988 argument with ACT*. That argument was that a theory should not get the power law of practice by building it into a mechanism. The graph-dynamic model meets that standard, but the curve it yields in its cleanest regime is logarithmic, and the evidence for the power law has itself been contested [16]. The defensible claim now is narrower and, I think, more interesting: that improvement with diminishing relative returns, which is what the data most clearly show, emerges from search dynamics without any learning curve being assumed, and that the same dynamics, with a mediocre decision procedure, make practice hurt.

Limitations. The game results are for three and four bits, and for one learning step at a time; the 1988 simulations used five bits and a hundred trials of learning. Reproducing Figure 1 of that paper by simulation, with the sampler checked against the exact small cases, is the next step. The tree idealization is stated, not derived from random-graph theory [17], and the exact fan-effect results in the theory use settings where the diameter is an integer. Nothing here concerns people: the phenomena are properties of the model's behavior, and connecting them to human practice data needs the usual assumptions about what is measured.

7 Conclusion

The 1988 paper argued that learning need not improve performance, and that whether it does depends on how decisions are made. Revisiting it with a proof assistant confirms that argument and corrects two of its supporting claims, and the corrections came from writing the theory down, not from new mathematics. That is the point of the title. A model that only runs can show that a behavior occurs; a model one can reason about shows what it entails, where its approximations hold, and which of its idealizations is responsible for a result. Thirty-eight years on, the graph-dynamic model has more to say than its authors knew.

Availability

All definitions and theorems are in SHH88Revisited [14] (Lean 4.35.0, Mathlib v4.35.0). Every theorem is fully proved, with no `sorry`, and depends only on Lean's standard axioms. The Python prototypes in `tools/` have the same semantics as the Lean definitions and generate the exact certificates.

Acknowledgements

The original work was done with Tad Hogg and Bernardo Huberman at Xerox PARC. Almost all of the Lean programming was accomplished by Claude Code with extensive guidance from the author. Papers and presentations were initially drafted by LLMs and then substantially rewritten by the author. Any remaining errors, including hallucinations, are the author's alone.

References

[1] George S. Snoddy. Learning and stability. *Journal of Applied Psychology*, 10:1–36, 1926.

[2] E. R. F. W. Crossman. A theory of the acquisition of speed-skill. *Ergonomics*, 2:153–166, 1959.

[3] Allen Newell and Paul S. Rosenbloom. Mechanisms of skill acquisition and the law of practice. In John R. Anderson, editor, *Cognitive Skills and Their Acquisition*, pages 1–55. Erlbaum, Hillsdale, NJ, 1981.

[4] John R. Anderson. *The Architecture of Cognition*. Harvard University Press, Cambridge, MA, 1983.

[5] Paul S. Rosenbloom. The chunking of goal hierarchies. In John Laird, Paul Rosenbloom, and Allen Newell, editors, *Universal Subgoaling and Chunking*. Kluwer, Boston, 1986.

[6] Jeff Shrager, Tad Hogg, and Bernardo A. Huberman. A graph-dynamic model of the power law of practice and the problem-solving fan-effect. *Science*, 242(4877):414–416, 1988.

[7] John R. Anderson. Retrieval of propositional information from long-term memory. *Cognitive Psychology*, 6:451–474, 1974.

[8] Steven Minton. Selectively generalizing plans for problem-solving. In *Proceedings of the Ninth International Joint Conference on Artificial Intelligence*, pages 596–599, 1985.

[9] Shaul Markovitch and Paul D. Scott. The role of forgetting in learning. In *Proceedings of the Fifth International Conference on Machine Learning*, pages 459–465, 1988.

[10] Jeff Shrager. CogLean: A library of cognitive theory in Lean. `https://github.com/jeffshrager/CogLean`, 2026.

[11] Jeff Shrager. CogLean-Intro: An invitation to CogLean. `https://github.com/jeffshrager/CogLean-Intro`, 2026.

[12] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.

[13] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, pages 367–381, 2020.

[14] Jeff Shrager. SHH88Revisited: Shrager, Hogg and Huberman (1988) in Lean. `https://github.com/jeffshrager/SHH88Revisited`, 2026.

[15] N. G. van Kampen. *Stochastic Processes in Physics and Chemistry*. North-Holland, Amsterdam, 1981.

[16] Andrew Heathcote, Scott Brown, and D. J. K. Mewhort. The power law repealed: The case for an exponential law of practice. *Psychonomic Bulletin & Review*, 7(2):185–207, 2000.

[17] Béla Bollobás. *Random Graphs*. Academic Press, London, 1985.